

Automated Report Delivery System

Setup guide — what it is built with, and how to start from scratch

Read this first. The full explanation of every file is in *ARD_Project_Report.pdf*.

31 August 2026

1. What is used

The project deliberately uses very little. Two Python libraries, one database, and one container image — everything else is part of Python itself.

Piece	Version	What it is for
Python	3.11 or newer (3.14.3 here)	The language everything is written in. Nothing older than 3.11 will work, because the code uses the modern <code>str None</code> type syntax.
PostgreSQL	16	The database holding the six ARD tables. Runs inside Docker, so it never has to be installed on the machine itself.
Docker Desktop	any current version (29.7.2 here)	Runs the PostgreSQL container. This is the only thing that has to be installed by hand besides Python.
psycopg[binary]	3.2.3 – 3.x (3.3.4 here)	The PostgreSQL driver: carries SQL to the database and brings the results back. [binary] means the ready-compiled build, so nothing needs a C compiler.
python-dotenv	1.x (1.2.3 here)	Reads the settings out of the <code>.env</code> file.
Tkinter	bundled with Python	The desktop window. It ships with the standard Windows Python installer, so there is nothing to install.

Parts of Python used with no installation at all

`csv` writes the report files · `smtp`lib and `email` build and send the mail with its attachments · `threading` and `queue` keep the window responsive while the scheduler runs · `logging` reports what is happening · `argparse` reads the command-line options · `datetime`, `calendar` and `pathlib` handle times, month lengths and file paths.

There is no web framework, no ORM and no scheduling library. Rows come back from the database as plain dictionaries, and the timer is a loop that sleeps.

2. What you get

Folder	What it contains
C:\project_ard	The pipeline alone, run from the command line. No window.

Folder	What it contains
C:\project_ard\ARD_GUI	The same pipeline plus a desktop window with six tabs. Start here unless you specifically want the command-line version.

ARD_GUI\	
gui.py	start the window
requirements.txt	the two libraries to install
docker-compose.yml	the PostgreSQL container
.env	your settings (database, timer, mail)
.env.example	a template of the same
ard\	the pipeline: config, db, csv_export, mailer,
	runner, scheduler, crud, __main__
ard\gui\	the window: app, spec, crud_tab, run_tab
sql\schema.sql	creates the six tables
sql\seed_data.sql	dummy data for testing
output\	CSV files appear here
logs\	ard.log and dry-run e-mails appear here

3. Starting from scratch on a new machine

Step 1 — install the two prerequisites

- **Python 3.11 or newer** from python.org. On Windows, tick *Add python.exe to PATH* during installation, and leave *tcl/tk and IDLE* ticked — that is Tkinter, and the window will not open without it.
- **Docker Desktop** from docker.com. Start it and wait until the whale icon says it is running.

Check both, in a terminal opened in the project folder:

```
python --version
docker --version
```

Step 2 — install the libraries

```
cd C:\project_ard\ARD_GUI
pip install -r requirements.txt
```

If the machine has more than one Python, use the one that answered in step 1. `python -m pip install -r requirements.txt` guarantees the libraries land in that same Python rather than another one.

Step 3 — start the database

```
docker compose up -d
```

This downloads PostgreSQL 16 the first time — a few minutes — and starts a container called `ard-postgres` on port 5432, with database, user and password all set up from `docker-compose.yml`. To confirm it is ready:

```
docker ps
```

The line for `ard-postgres` should say `(healthy)`. If it still says `(starting)`, wait a few seconds and look again.

Step 4 — check the settings

If there is no `.env` file in the folder, copy the template:

```
copy .env.example .env
```

The defaults already match the container, so nothing has to be changed to get started. The two settings worth knowing about:

Setting	Meaning
POLL_INTERVAL_MINUTES	How often the program checks the schedule. 30 by default; set it to 1 or 2 while testing so you do not have to wait.

Setting	Meaning
SMTP_DRY_RUN	true (the default) writes each e-mail to <code>logs*.eml</code> instead of sending it, so you can test safely. Set it to false and fill in the SMTP_USER and SMTP_PASSWORD lines to send for real.

Step 5 — create the tables and load the test data

```
python -m ard --init-db --seed
```

It should print `applied schema.sql` and `applied seed_data.sql`. The database now holds the six tables plus four sample tasks, seven queries, five schedules and eight recipients. Running this command again at any time wipes everything and starts over.

Step 6 — open the window

```
python gui.py
```

Six tabs appear: Task, Query, Schedule, Email config, Recipient, and Run and log. If a pop-up says the database cannot be reached, go back to step 3.

Step 7 — prove it works

- Go to the **6. Run and log** tab.
- Choose *1 - Daily Transactions* in the dropdown and press **Run task**.
- The output pane should show three CSV files being written and the task being delivered to 3 recipients.
- The execution log above fills with three rows, all reading SENT with a delivery time.
- Look in the output folder for the three CSV files, and in `logs` for the `.eml` file — open it in any mail program to see exactly what would have been sent.

Task 1 produces three files because it has three active queries. That is the rule from the handwritten notes: one task, several queries, several files, all named from the task's output prefix.

4. Using it for real

Setting up your own report

Work through the tabs left to right — the order is deliberate, because each tab needs the one before it.

	Tab	What to do
1	Task	Press New , type a name and an output prefix (the start of the CSV filename), press Save .
2	Query	New , choose your task, paste the SQL, press Save . Add a second query for the same task if you want a second file.
3	Schedule	New , choose the task, set the time as HH:MM, pick DAILY / WEEKLY / MONTHLY. For WEEKLY put a weekday name in Day spec; for MONTHLY a day number, or the word LAST.
4	Email config	New , choose the task, write the subject, body and signature. One per task.
5	Recipient	New , choose the e-mail config, add an address and pick TO, CC or BCC. Repeat for each person.
6	Run and log	Press Start scheduler . It now checks every interval and delivers by itself.

Turning something off without losing it

Every one of these has an **Active** tick box. Unticking it stops the item being used from now on while keeping it, and its history, in the database. Untick a recipient who has left, a query version you have replaced, or a schedule you want to pause.

Switching on real e-mail

- Open `.env` and set `SMTP_DRY_RUN=false`.
- Fill in `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD` and `SMTP_FROM`.
- For Gmail this must be an app password, not the account password.
- Restart the program — settings are read once at start-up.

Running without the window

Command	What it does
<code>python -m ard</code>	Runs the scheduler in the terminal until you stop it with Ctrl+C.
<code>python -m ard --poll-once</code>	One check, then exits.
<code>python -m ard --run-task 1</code>	Runs task 1 immediately, ignoring its schedule.
<code>python -m ard --status</code>	Prints the last 20 execution-log entries.
<code>python -m ard --gui</code>	Opens the window — the same as <code>python gui.py</code> .

5. If something does not work

What you see	What to do
<code>ModuleNotFoundError: No module named 'psycpg'</code>	The libraries went into a different Python than the one you are running. Install them into the right one with <code>python -m pip install -r requirements.txt</code> .
connection refused, or the window's "Cannot reach the database" pop-up	The container is not running. Start Docker Desktop, then <code>docker compose up -d</code> , and check <code>docker ps</code> says healthy.
relation "task" does not exist	The tables were never created. Run <code>python -m ard --init-db --seed</code> .
No matching distribution found when installing	The Python version is too old, or too new for a pinned library. <code>requirements.txt</code> uses ranges to avoid this — make sure you are installing from that file rather than naming a version by hand.
The tick says a schedule was found, but nothing runs	The log line reads <i>"already ran for ..."</i> . A schedule fires once per due time. Move its time forward, or clear that schedule's rows from <code>EXECUTION_LOG</code> while testing.
No e-mail arrives	Check whether <code>SMTP_DRY_RUN</code> is still true — in that case the message is waiting in <code>logs*.eml</code> , which is exactly what should happen until you switch it off.
The window will not open at all	Tkinter is missing. Re-run the Python installer, choose Modify, and tick <i>tcl/tk and IDLE</i> .

Starting completely over

```
docker compose down -v
docker compose up -d
python -m ard --init-db --seed
```

The `-v` deletes the stored data as well as the container, so this returns the machine to a fresh install.